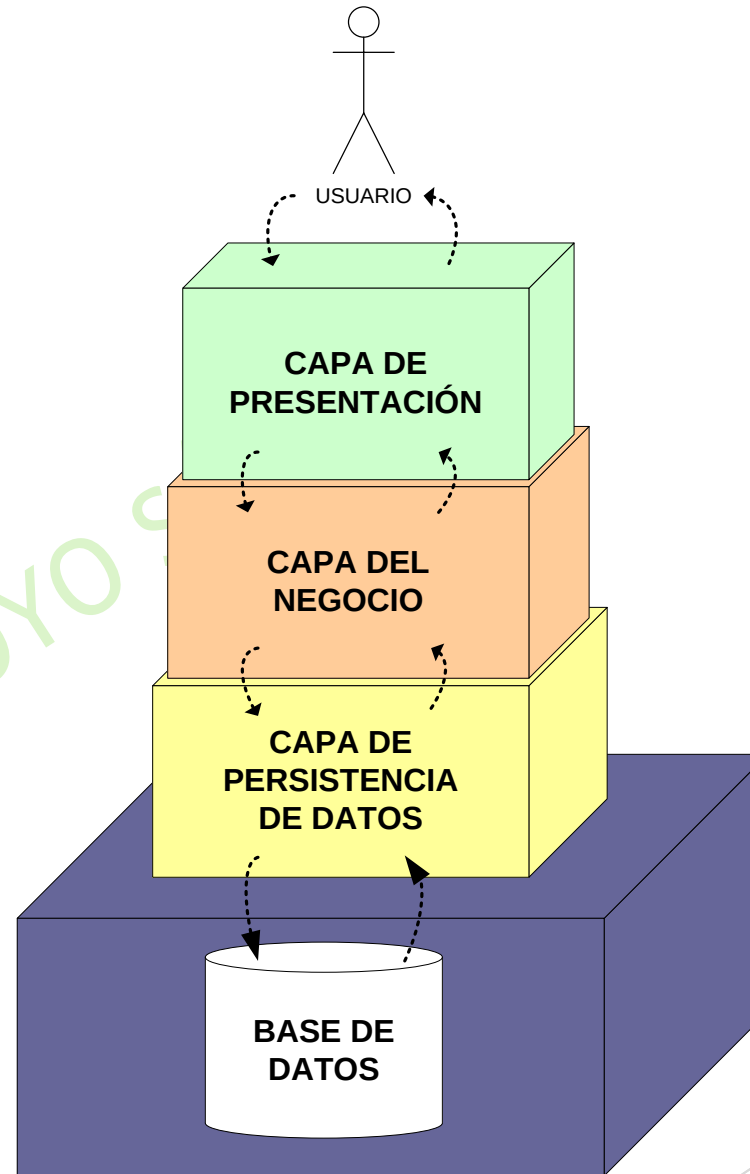


UD 3 - MEJORANDO NUESTRAS CLASES



UD 3 - MEJORANDO NUESTRAS CLASES

Índice

1. El cambio como constante.
Arquitectura de una aplicación
2. Encapsulamiento
3. Constructores
4. Sobrecarga de métodos
5. Asertos
6. El modificador static

UD 3 - MEJORANDO NUESTRAS CLASES

1 - El cambio como constante. Arquitectura de una aplicación

► 1 – El cambio como constante. Arquitectura de una aplicación

- En el mundo del desarrollo de software solo existe una **CONSTANTE**:

“TODO CAMBIA”

- Si tu aplicación no funciona bien o no hace lo que el usuario necesita está claro que **HABRÁ QUE CAMBIARLA**.
- Pero si tu aplicación funciona y satisface las necesidades del usuario entonces es muy probable que se pidan funcionalidades nuevas o que haya que hacerla más segura o actualizar las dependencias... vamos que sí o sí **HABRÁ QUE CAMBIARLA**.

¿Piensas que la página de Facebook que ves a día de hoy es la misma que se publicó en su primera versión?



UD 3 - MEJORANDO NUESTRAS CLASES

1 - El cambio como constante. Arquitectura de una aplicación

- ▶ Como desarrolladores de software tenemos que **MENTALIZARNOS DE QUE EL CAMBIO ES INEVITABLE**, buscando siempre “nadar a favor de la corriente” y nunca en contra. Esto significa que tendremos que tomar ciertas medidas en nuestro código para poder “gestionar correctamente” el cambio.
- ▶ La arquitectura de la aplicación, la adopción de una metodología Agile, los patrones de diseño, la refactorización del código para que se “lea” mejor, usar un criterio uniforme de nombrado, el uso de un repositorio de código como Git, la automatización de las pruebas... son técnicas que ayudan a “soportar mejor el cambio”.
- ▶ Todas estas técnicas buscan **mejorar la legibilidad del código y desacoplar las distintas partes del sistema** para que, si tenemos que modificar una de ellas, el impacto en el resto de piezas sea el menor posible y se realice con el menor coste posible.



UD 3 - MEJORANDO NUESTRAS CLASES

1 - El cambio como constante. Arquitectura de una aplicación

- Algunas técnicas para la gestión del cambio las estudiaréis en este módulo, otras en el módulo de Entornos de Desarrollo, otras en la empresa... La cuestión es que es vital incorporarlas en la práctica diaria para poder siempre “nadar a favor de corriente”.

Arquitectura de una aplicación

- Antiguamente, se realizaba un desarrollo **MONOLÍTICO**, “todo en un bloque”, en el que todo el código de la aplicación estaba en un solo fichero. Esto provocaba un código difícil de manejar, de mantener, de localizar y solucionar los errores... **Un código que no estaba preparado para el cambio.**
- Actualmente, un proyecto de software se basa en una **arquitectura en capas**. Esto divide el software en partes más pequeñas y manejables. Además favorece el trabajo simultáneo de distintos equipos de trabajo.



UD 3 - MEJORANDO NUESTRAS CLASES

1 - El cambio como constante. Arquitectura de una aplicación

- ▶ Esta forma de estructurar el software nos lleva a la metáfora de un **edificio donde cada planta es como una capa del software.**
 - ▶ Aunque esta metáfora no es 100% representativa porque podríamos empezar a construir las dos capas de arriba del software y todavía no tener los cimientos puestos...
 - ▶ La metáfora de la confección de un traje complejo me parece más acertada. Tenemos que juntar distintas piezas... “Probárselo” al cliente... Empezar por la parte de arriba/abajo, lo de dentro/fuera...
- ▶ En cualquier caso, **cada capa del software debe:**
 - ▶ Tener una responsabilidad BIEN DEFINIDA.
 - ▶ Interactuar sólo con las capas ADYACENTES.



UD 3 - MEJORANDO NUESTRAS CLASES

1 - El cambio como constante. Arquitectura de una aplicación

► Una posible arquitectura de tres capas:

► La capa de presentación se responsabiliza de:

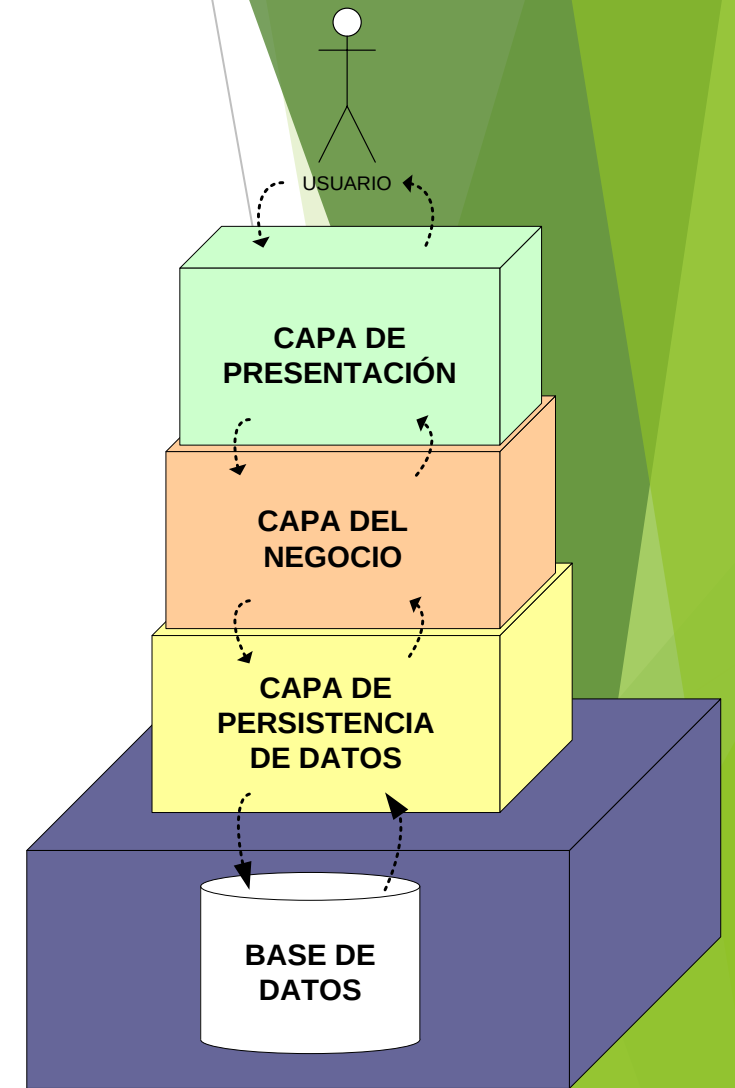
- Atender las peticiones del usuario y las traslada a la capa de negocio.
- Presentar al usuario los resultados que le devuelve la capa de negocio.

► La capa del negocio se responsabiliza de:

- Recibir las peticiones de la capa de presentación y realizar los cálculos/operaciones propias del negocio que se esté modelando, utilizando la capa de persistencia si es necesario.
- A esta capa también se le llama **capa de dominio**.

► La capa de persistencia se responsabiliza de:

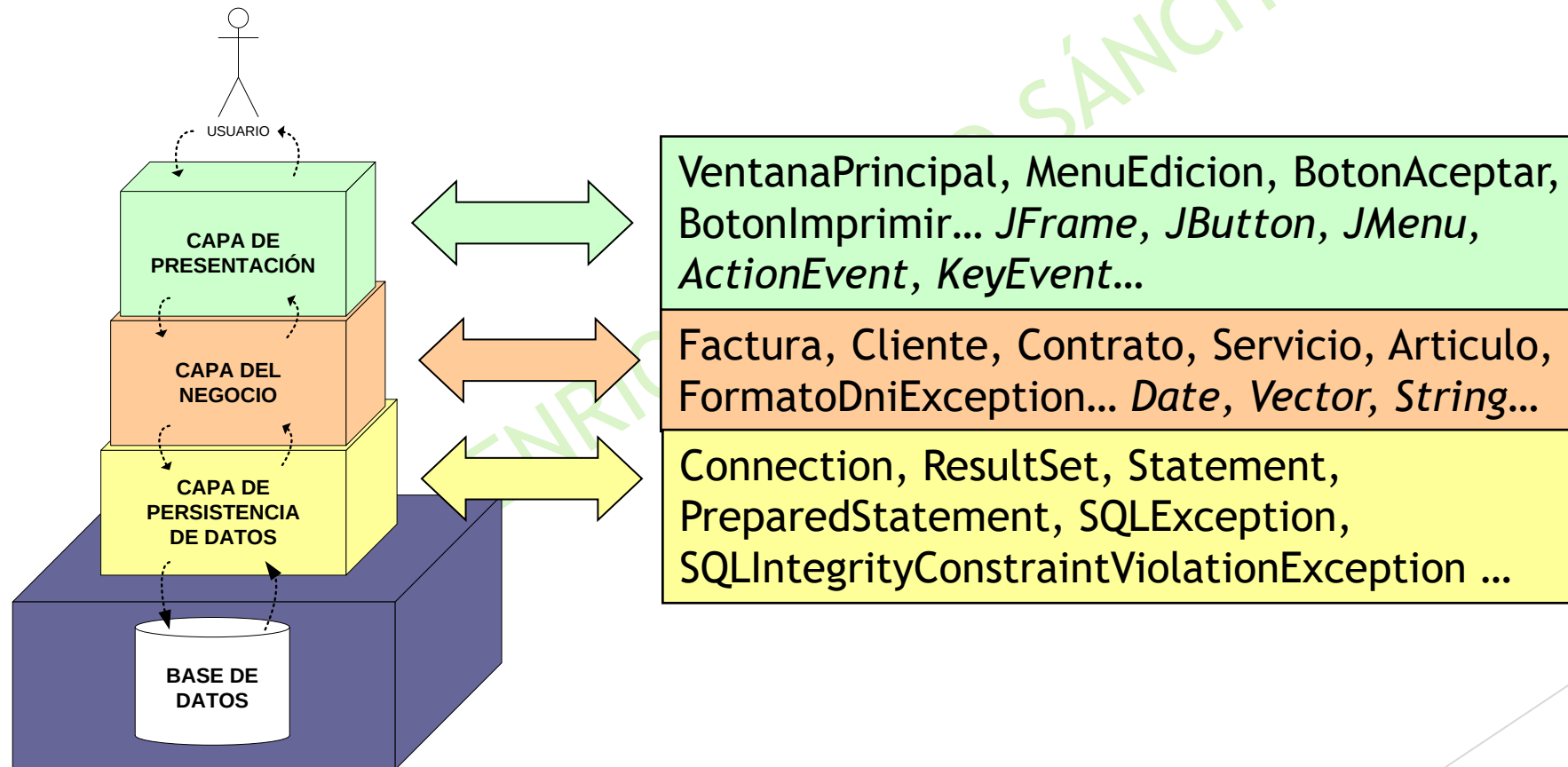
- Guardar y recuperar los objetos de la capa de negocio en la base de datos, realizando las adaptaciones necesarias para ello.



UD 3 - MEJORANDO NUESTRAS CLASES

1 - El cambio como constante. Arquitectura de una aplicación

- En cada capa se crearán los objetos que sean necesarios para realizar la operación requerida por el usuario. Los objetos irán creándose, interactuando entre ellos y muriendo según vaya haciendo falta.

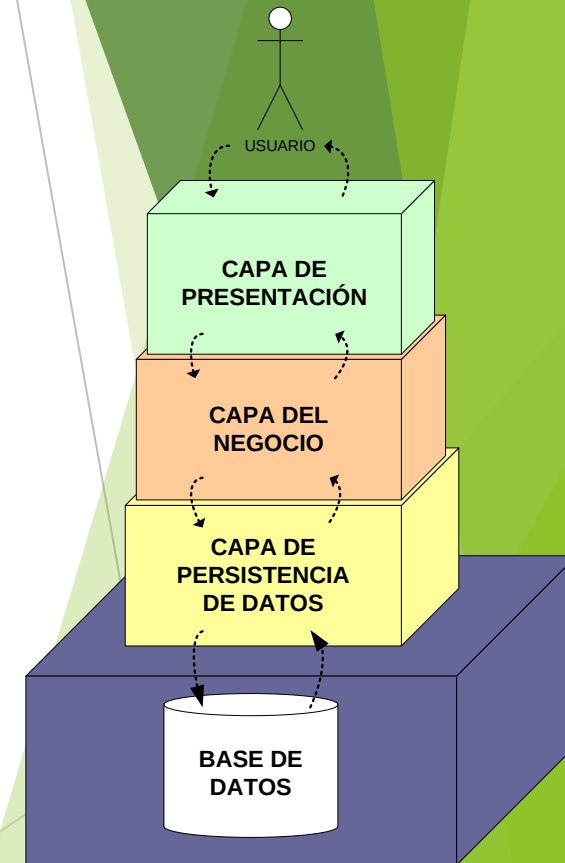


Pueden crearse tantas capas como sean necesarias (normalmente 3 o más)

UD 3 - MEJORANDO NUESTRAS CLASES

1 - El cambio como constante. Arquitectura de una aplicación

- ▶ Cada capa “**ofrece sus servicios**” a la capa superior y “**utiliza los servicios**” de la capa inferior. De este modo vamos ganando capacidades a medida que subimos por las capas.
- ▶ Ejemplo con una web de banca online:
 - ▶ La capa de presentación ofrece al usuario un **servicio de interacción con la aplicación**. Aquí podríamos hablar de objetos ventana, botón, lista desplegable, menús...
 - ▶ La capa de negocio ofrece a la capa de presentación **el servicio de realización de las operaciones bancarias** solicitadas por el usuario (transferencias, ingresos, pago de recibos...).
 - ▶ La capa de persistencia de datos ofrece a la capa de negocio **el servicio de almacenamiento y recuperación de la información** requerida para la realización de las operaciones bancarias (consulta los movimientos bancarios en la BD, añade un movimiento más al sistema...)



UD 3 - MEJORANDO NUESTRAS CLASES

1 - El cambio como constante. Arquitectura de una aplicación

- ▶ En esta unidad didáctica y en la siguiente **NO** vamos a realizar software por capas.
- ▶ Vamos a estudiar distintas herramientas de la orientación a objetos que se utilizan habitualmente para crear cada una de las capas para que sean “**resistentes al cambio**”.

Encapsulamiento

Sobrecarga de métodos

Herencia

Sobreescritura de métodos

Modificador static

CONSTRUCTORES

Composición

POLIMORFISMO

Clases abstractas

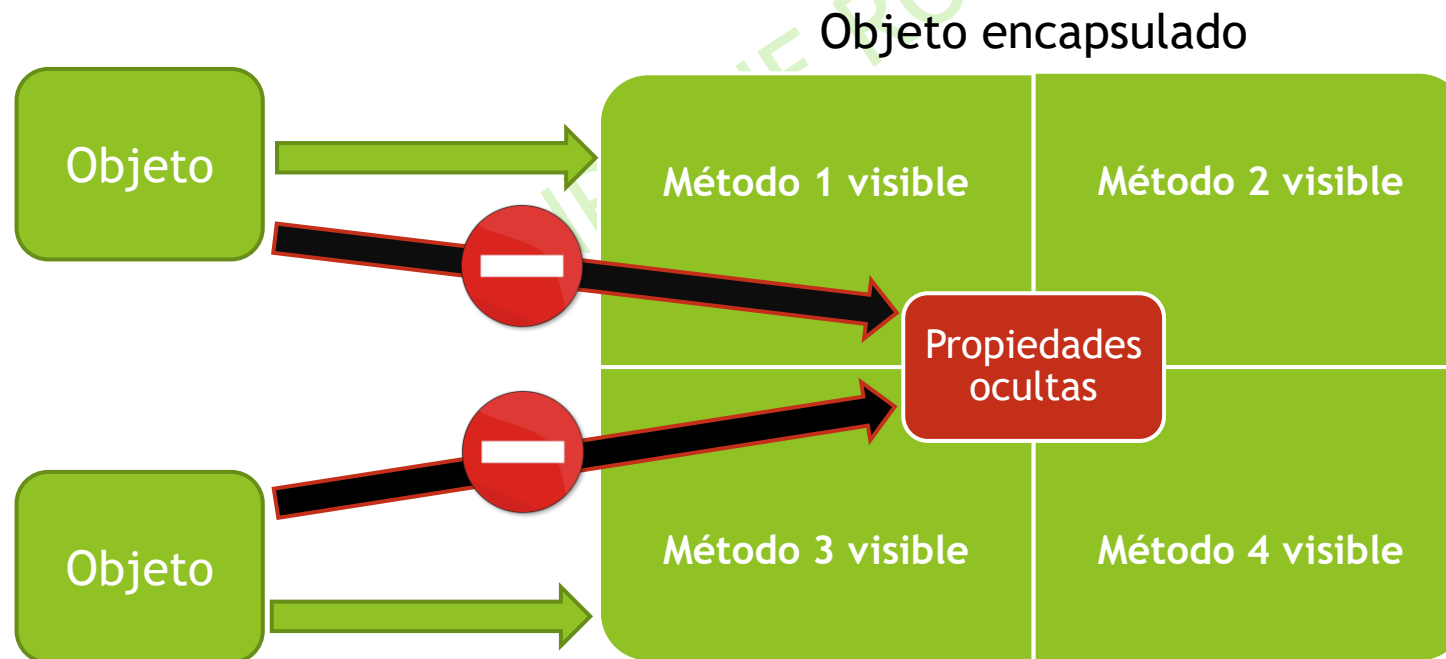
INTERFACES

UD 3 - MEJORANDO NUESTRAS CLASES

2 - Encapsulamiento

► 2 – Encapsulamiento

- El encapsulamiento es una técnica de diseño de clases que consiste en **ocultar el estado interno del objeto** (sus propiedades) y mostrar sólo los métodos.
- Es como si el objeto tuviera “**SUS SECRETOS**”, de modo que los objetos externos sólo pueden hacer uso de los métodos y nunca acceder directamente a las propiedades del objeto.



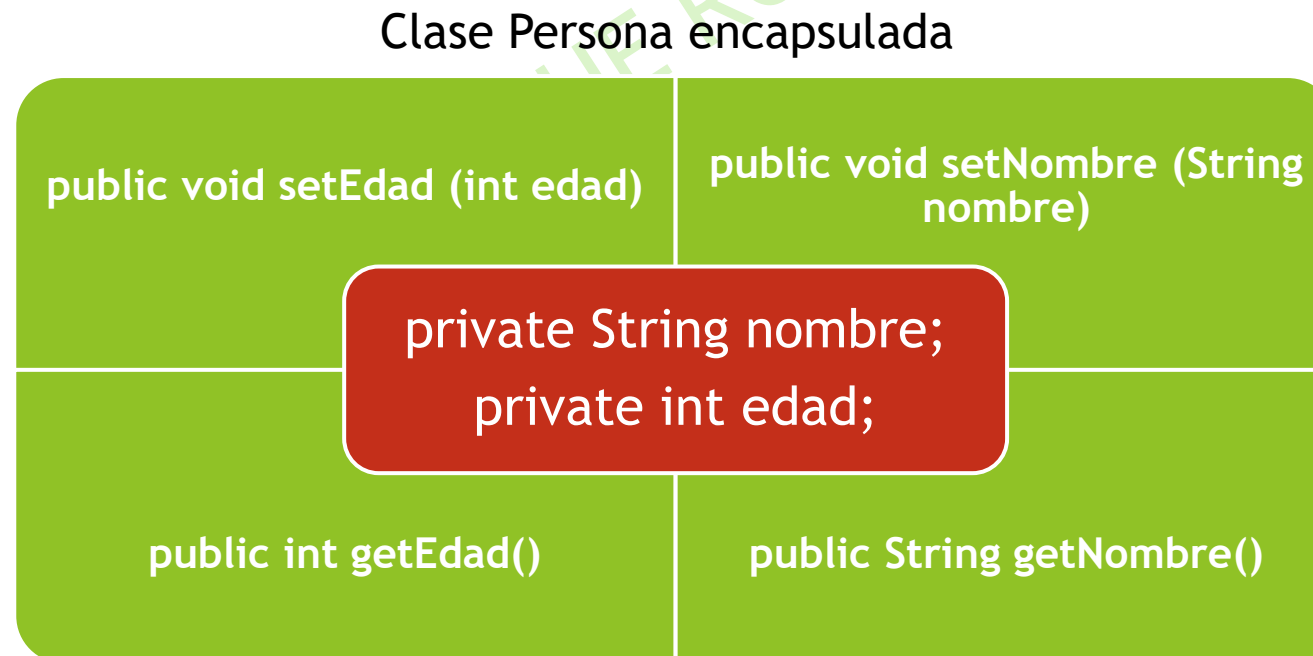
UD 3 - MEJORANDO NUESTRAS CLASES

2 - Encapsulamiento

- Para realizar el encapsulamiento de una clase necesitamos usar los modificadores de acceso del lenguaje, que son:



- Normalmente, se usan **public** y **private**. Un ejemplo:



protected y
"friendly" los
estudiaremos un
poco más
adelante

Si hacemos que
nuestras
propiedades sean
private entonces
SOLO podrán
accederse desde
dentro de la clase
que las contiene

UD 3 - MEJORANDO NUESTRAS CLASES

2 - Encapsulamiento

- ▶ Hasta hoy podíamos “hackear” nuestros objetos desde fuera y saltarnos las validaciones que hacemos en los métodos. Ejemplo:

```
Bombilla b = new Bombilla(); // Nuestra bombilla SE FUNDE en su encendido nº 10
```

```
for (int i=1; i<=20; i++)
```

```
{
```

```
    b.apagar();
```

```
    b.encender();
```

```
}
```

```
if (b.fundida == true)
```

```
    b.fundida = false;
```

OBJETO HACKEADO.
Cambiamos el estado
interno a nuestra
conveniencia.



- ▶ El encapsulamiento impide que un objeto externo pueda manipular directamente el estado de nuestro objeto dejándolo en un estado incoherente. De este modo, conseguimos que nuestros objetos sean más robustos y confiables.

Descargamos del
repositorio
U3.P1.Encapsulamiento
y lo probamos.

UD 3 - MEJORANDO NUESTRAS CLASES

2 - Encapsulamiento

► A PARTIR DE HOY, TODAS NUESTRAS CLASES DEBEN ENCAPSULARSE.

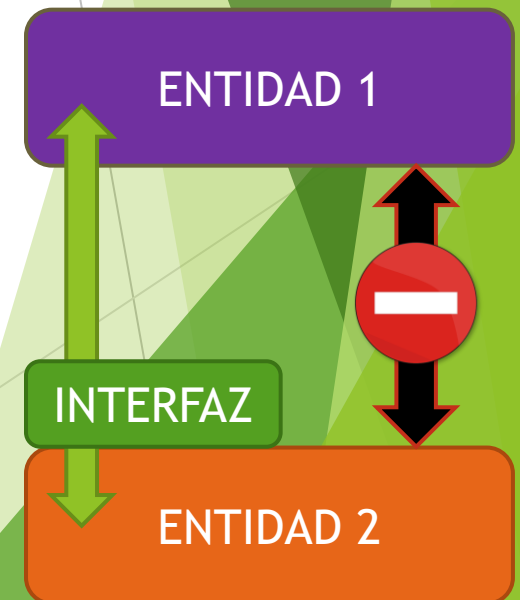
- Solo permitiremos el uso de propiedades públicas para crear constantes estáticas que puedan ser accedidas desde fuera. Ejemplo: Math.PI

► Concepto de interfaz de una clase/objeto:

- En general, una interfaz es un mecanismo de interacción o comunicación entre dos entidades que no sabrían “hablar” entre ellas directamente.

► Ejemplos:

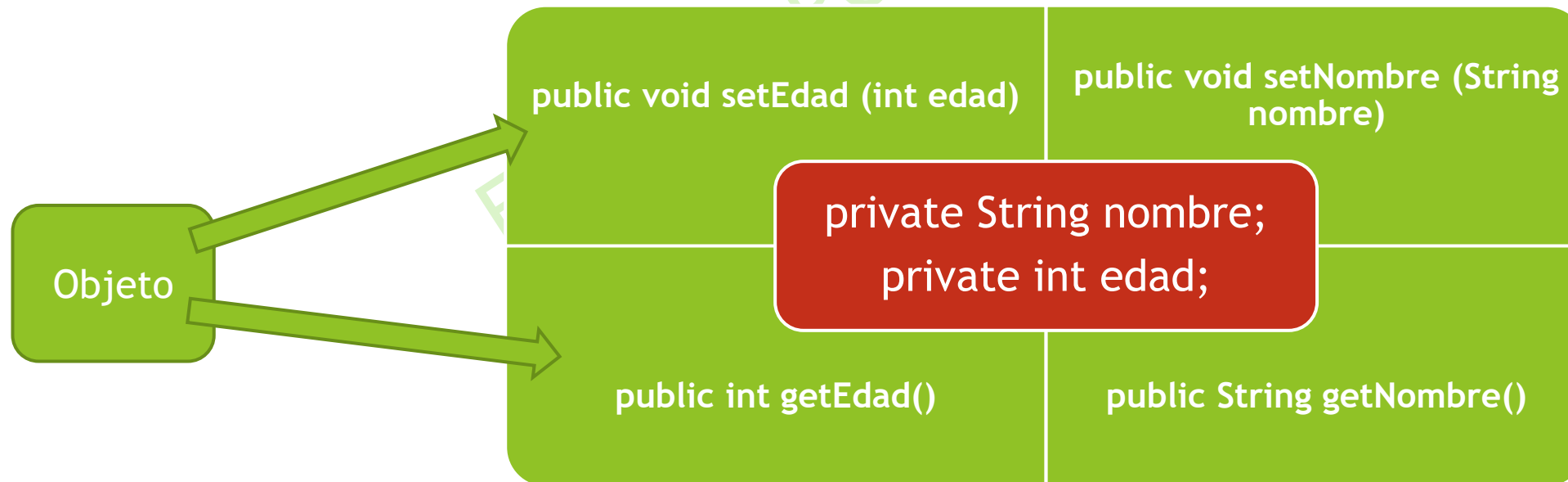
- El volante, los pedales, la palanca de cambio, el cuenta-revoluciones... todos estos elementos componen la INTERFAZ DE COMUNICACIÓN entre un humano y un vehículo.
- Una ventana, el puntero del ratón, botones, menús... todos estos elementos componen la INTERFAZ GRÁFICA DE COMUNICACIÓN entre un humano y una aplicación.



UD 3 - MEJORANDO NUESTRAS CLASES

2 - Encapsulamiento

- ▶ Con el encapsulamiento, nuestros objetos **ocultan sus propiedades** y ofrecen al resto de objetos **una interfaz de acceso** a ellas mediante su lista de métodos públicos.
- ▶ Podemos decir entonces que un objeto/clase ofrece **un conjunto de servicios** al resto de objetos mediante su **interfaz de métodos públicos**.

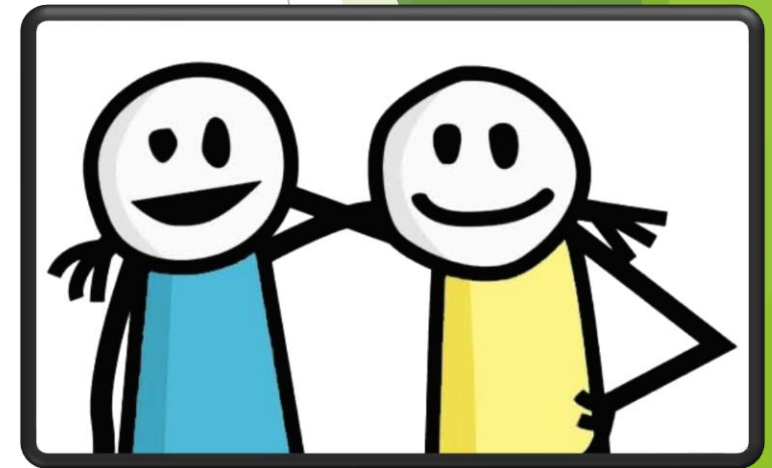


UD 3 - MEJORANDO NUESTRAS CLASES

2 - Encapsulamiento

- ▶ El modificador de acceso de paquete o “friendly”:
 - ▶ Si a una propiedad no se le pone ningún modificador de acceso (public, private...) entonces se dice que tiene accesibilidad “a nivel de paquete” o “friendly” (para los amigos o vecinos). Esto haría que esa propiedad se comportara como:
 - ▶ Public para las clases que estén en su mismo paquete.
 - ▶ Private para el resto de las clases.
 - ▶ Ejemplo:

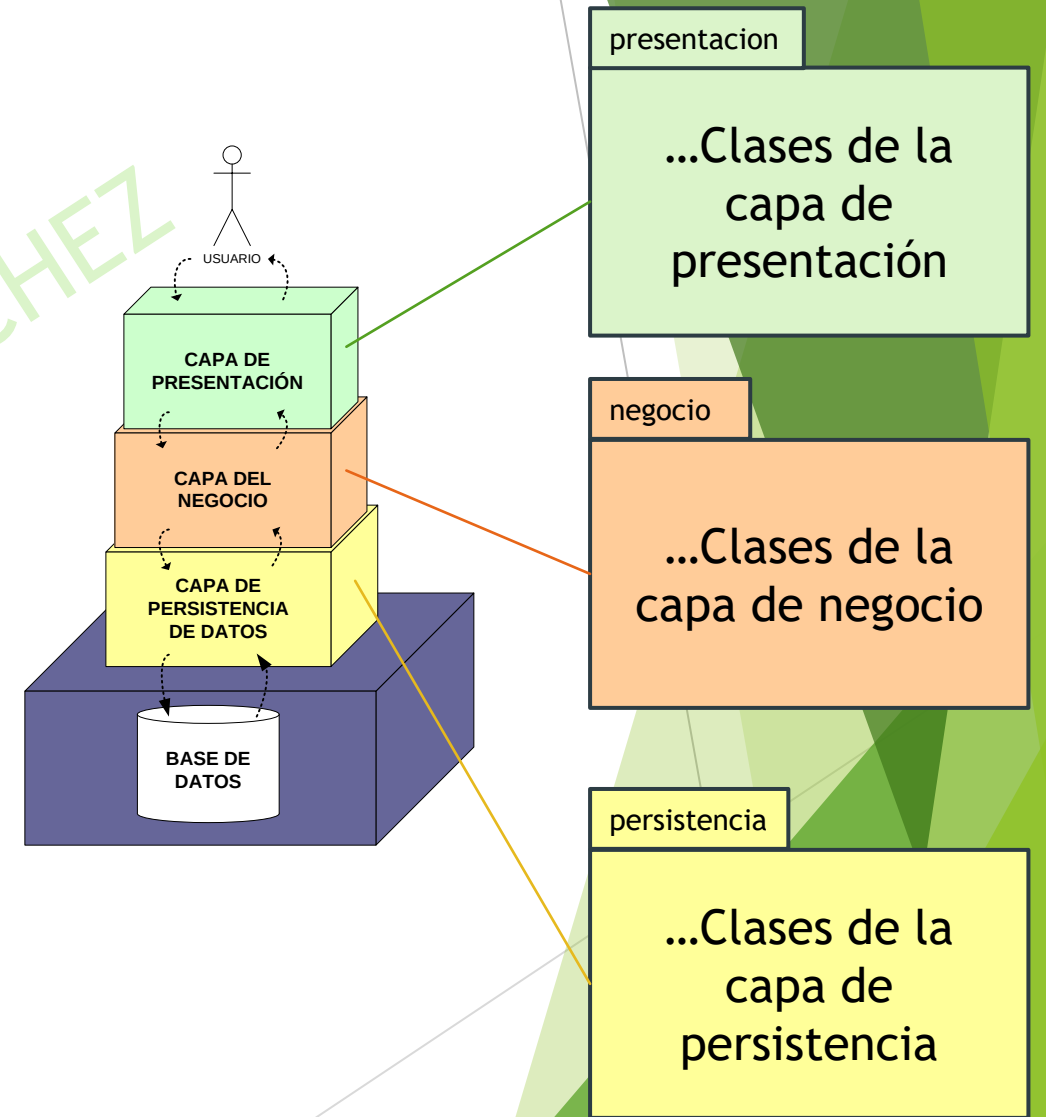
```
public class Persona {  
    String nombre;  
    int edad;  
    //...  
}
```



UD 3 - MEJORANDO NUESTRAS CLASES

2 - Encapsulamiento

- ▶ Normalmente las clases que pertenecen a un mismo paquete:
 - ▶ Están relacionadas entre sí.
 - ▶ Suelen cooperar entre ellas.
 - ▶ Han sido desarrolladas por un mismo equipo de desarrolladores.
- ▶ Todo esto nos lleva a establecer una relación de “vecindad” o “confianza” entre estas clases.
- ▶ El modificador “friendly” permite que ciertas propiedades/métodos sean accesibles para las “clases vecinas” pero no para el resto del mundo.



UD 3 - MEJORANDO NUESTRAS CLASES

2 - Encapsulamiento

- ▶ El concepto de “encapsulamiento” también se puede aplicar a nivel de paquete.
 - ▶ En este caso nos interesa que algunas clases de un paquete sean accesibles “desde fuera” y otras no.
 - ▶ Cuando creamos una clase dentro de un paquete tenemos dos opciones:
 - ▶ Ponerla como **public**, de modo que sería visible “desde fuera” del paquete que la contiene.
 - ▶ Ponerla como “**friendly**”, de modo que sería visible por las clases que estén en su mismo paquete, pero no “desde fuera”.

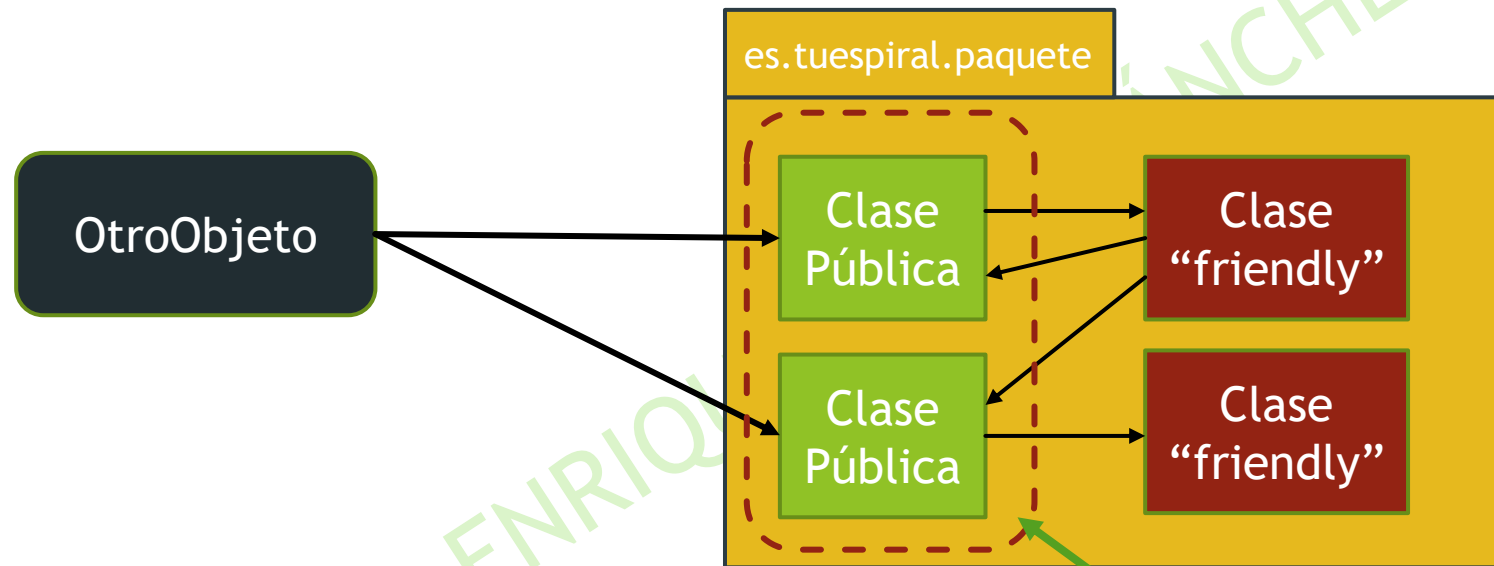
```
package es.tuespiral.paquete;  
public class PersonaPublica {  
    String nombre;  
    int edad;  
    ...  
}
```

```
package es.tuespiral.paquete;  
class PersonaPaquete {  
    String nombre;  
    int edad;  
    ...  
}
```


UD 3 - MEJORANDO NUESTRAS CLASES

2 - Encapsulamiento

- De este modo conseguimos que ciertas clases de un paquete sean públicas y “den la cara” y otras clases “se queden atrás” actuando como un apoyo o complemento a las clases públicas.



- Así que también podemos hablar de la **interfaz de un paquete** como el conjunto de clases públicas de dicho paquete.

UD 3 - MEJORANDO NUESTRAS CLASES

3 - Constructores

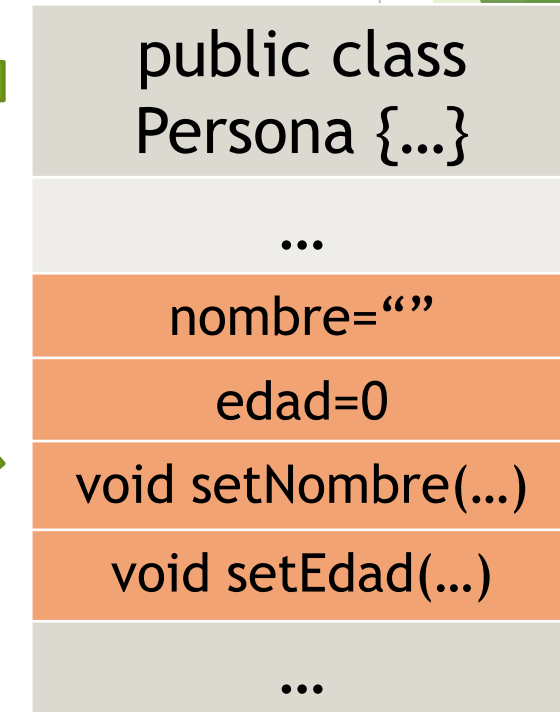
► 3 – Constructores

- Los constructores son métodos que nos permiten crear un objeto a partir de la clase.
- Los estamos usando cada vez que hacemos:
 - ... = **new** Persona();
 - ... = **new** Bombilla();
 - ... = **new** Scanner(System.in);
- Ahora vamos a aprender a crearlos y a estudiar las reglas con las que funcionan.
- Veamos un ejemplo:

Constructor



Memoria



Código de
la clase
Persona

Objeto de
la clase
Persona

UD 3 - MEJORANDO NUESTRAS CLASES

3 - Constructores

- Vamos a crear un constructor que nos permita inicializar todas las propiedades de un objeto Persona a la vez.

```
public class Persona {  
    private String nombre, apellidos;  
    private int edad, altura;  
    private boolean casada;  
  
    public Persona(String nombre, String apellidos, int edad, int altura, boolean casada) {  
        this.nombre = nombre;  
        this.apellidos = apellidos;  
        this.edad = edad;  
        this.altura = altura;  
        this.casada = casada;  
    }  
  
    // Getters y Setters  
    public String getNombre() {  
        return nombre;  
    }  
}
```

this sirve para que la clase hable de sí misma, de sus propiedades y métodos. Se le llama autorreferencia

UD 3 - MEJORANDO NUESTRAS CLASES

3 - Constructores

- Esto nos permite crear los objetos más cómodamente y los tenemos listos para usarse justo después de la llamada al constructor:

```
public class PruebaPersona {  
    public static void main(String[] args) {  
        Persona p = new Persona("Pedro", "López Haro", 25, 180, false);  
    }  
}
```

- Lo que hacíamos hasta ahora era mucho más largo y dejaba el objeto en un estado inicial incoherente o “no utilizable”:

Estado inicial
incoherente

Estado
“usable”
después de
los setters

```
Persona p = new Persona();  
p.setNombre("Pedro");  
p.setApellidos("López Haro");  
p.setEdad(25);  
p.setAltura(180);  
p.setCasada(false);
```

UD 3 - MEJORANDO NUESTRAS CLASES

3 - Constructores

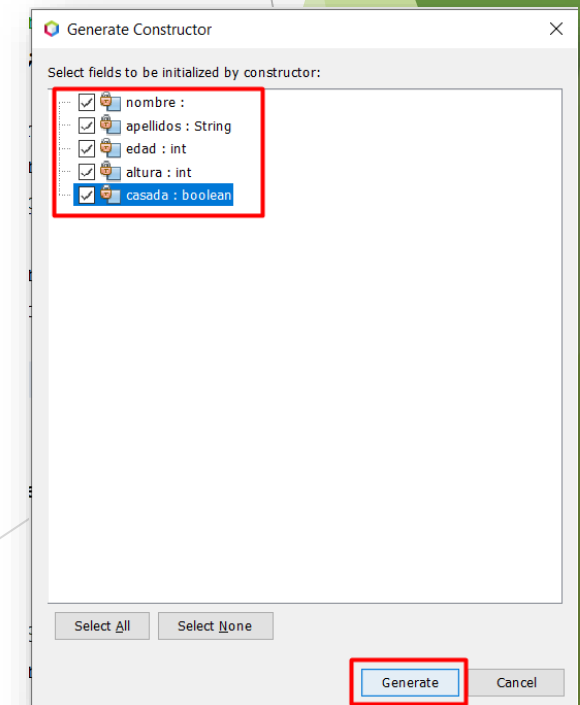
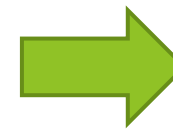
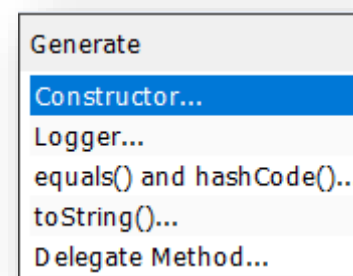
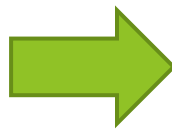
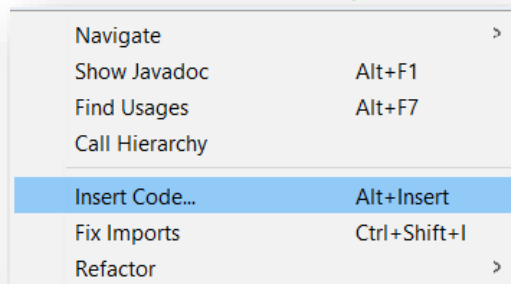
- ▶ La sintaxis de un constructor es la siguiente:

public NombreDeLaClase (lista de parámetros) { ... }

- ▶ Ejemplos:

- ▶ `public Bombilla (int potencia, String marca) { ... }`
- ▶ `public Triangulo (double lado1, double lado2, double lado3) { ... }`
- ▶ `public CajaRegistradora () { ... }` // Constructor sin parámetros

- ▶ NetBeans te lo pone fácil



UD 3 - MEJORANDO NUESTRAS CLASES

3 - Constructores

- ▶ **El constructor “por defecto”:** cuando el desarrollador NO CREA ningún constructor para una clase, el lenguaje Java crea automáticamente un constructor sin parámetros. Este es el constructor que hemos usado hasta ahora y se le llama constructor “por defecto”.
 - ▶ **Metáfora:** al acusado de un juicio le ponen un “abogado de oficio” solo si no lleva su propio abogado.
- ▶ **OJO:** si el desarrollador crea un constructor, Java ya no te crea el constructor “por defecto”... esto provoca errores porque estamos tan acostumbrados a usarlo que tendemos a pensar que siempre está ahí.

Descargamos del repositorio
U3.P2.Constructores y lo probamos.



UD 3 - MEJORANDO NUESTRAS CLASES

4 - Sobrecarga de métodos

► 4 – Sobrecarga de métodos

- Se llama sobrecarga de métodos a **crear varias versiones de un método variando los parámetros de entrada**. Ejemplo:

```
public class Persona {  
    private String nombre, apellidos;  
    private int edad, altura;  
    private boolean casada;  
  
    public Persona(String nombre, String apellidos, int edad, int altura, boolean casada) {  
        this.nombre = nombre;  
        this.apellidos = apellidos;  
        this.edad = edad;  
        this.altura = altura;  
        this.casada = casada;  
    }  
  
    public Persona(String nombre, String apellidos) {  
        this.nombre = nombre;  
        this.apellidos = apellidos;  
    }  
  
    // Getters y Setters
```

UD 3 - MEJORANDO NUESTRAS CLASES

4 - Sobrecarga de métodos

- Esto nos **ayuda a tener que recordar menos nombres de métodos**, disponiendo de distintas “versiones” que comparten el mismo nombre.
- **Podemos sobrecargar cualquier método**, ya sea un constructor o un método “normal”. 
- Para crear las distintas “versiones”, podemos variar el número y tipo de los parámetros de entrada, pero nunca cambiar el parámetro de salida. Si intentamos añadir el siguiente método a la clase de la imagen de la derecha el IDE nos daría un error:

► `public short suma (short a, short b)`

No habría forma de saber si quieres usar una versión u otra

```
public class Sobrecarga {  
  
    public int suma(int a, int b) {  
        return a+b;  
    }  
  
    public int suma(int a, int b, int c) {  
        return a+b+c;  
    }  
  
    public int suma(short a, short b) {  
        return a+b;  
    }  
  
    public int suma(short a, short b, short c) {  
        return a+b+c;  
    }  
}
```

Realizar los ejercicios 1 al 2 del boletín

UD 3 - MEJORANDO NUESTRAS CLASES

5 - Asertos

5 – Asertos

- Siempre decimos que una clase debe saber "**defenderse**" de los malos usos que se puedan hacer de ella. Hemos usado el `if` para conseguir este efecto escribiendo cosas como:

```
public void marcaGol (int numGoles) {  
    if (numGoles <= 0) {  
        System.out.println("Error: el número de goles debe ser un valor positivo");  
    } else {  
        this.numGoles = this.numGoles + numGoles;  
    }  
}
```

- Sin embargo, a medida que nuestra aplicación crece, este "mensajito" en la consola puede **pasar inadvertido y nuestro programa continúa** su ejecución como si nada. Esto puede provocar errores o comportamientos inesperados en otros puntos del código que sean **difíciles de detectar** (errores laterales).



UD 3 - MEJORANDO NUESTRAS CLASES

5 - Asertos

- Podríamos querer diseñar clases más "catastrofistas" que **paren la ejecución** de la aplicación si se detecta cualquier uso inapropiado. Este enfoque se le llama **FAIL FAST** y se busca que la aplicación "falle" en cuanto se detecte un error.
- Para conseguir este efecto, Java nos brinda la instrucción **assert** ("asegurar" o "asertar") que funciona de una forma muy sencilla:



```
public void marcaGol (int numGoles) {  
    assert (numGoles > 0) : "El número de goles debe ser un valor positivo";  
    this.numGoles = this.numGoles + numGoles;  
}
```

Es como si dijésemos:
"Asegúrate de que el número
de goles es > 0 antes de seguir"

- También podemos omitir el mensaje y los paréntesis:

```
public void marcaGol (int numGoles) {  
    assert (numGoles > 0);  
    this.numGoles = this.numGoles + numGoles;  
}
```

```
public void marcaGol (int numGoles) {  
    assert numGoles > 0;  
    this.numGoles = this.numGoles + numGoles;  
}
```

UD 3 - MEJORANDO NUESTRAS CLASES

5 - Asertos

- Sin embargo, si ejecutamos un *main* como el siguiente:

```
public class PruebaFutbolista {  
    public static void main(String[] args) {  
        Futbolista f = new Futbolista("Diego Armando", "Maradona");  
  
        f.marcaGol(-5);  
        System.out.println("Num goles maradona "+f.getNumGoles());  
    }  
}
```

observamos que el programa no se detiene y se mostrará:

Num goles maradona -5

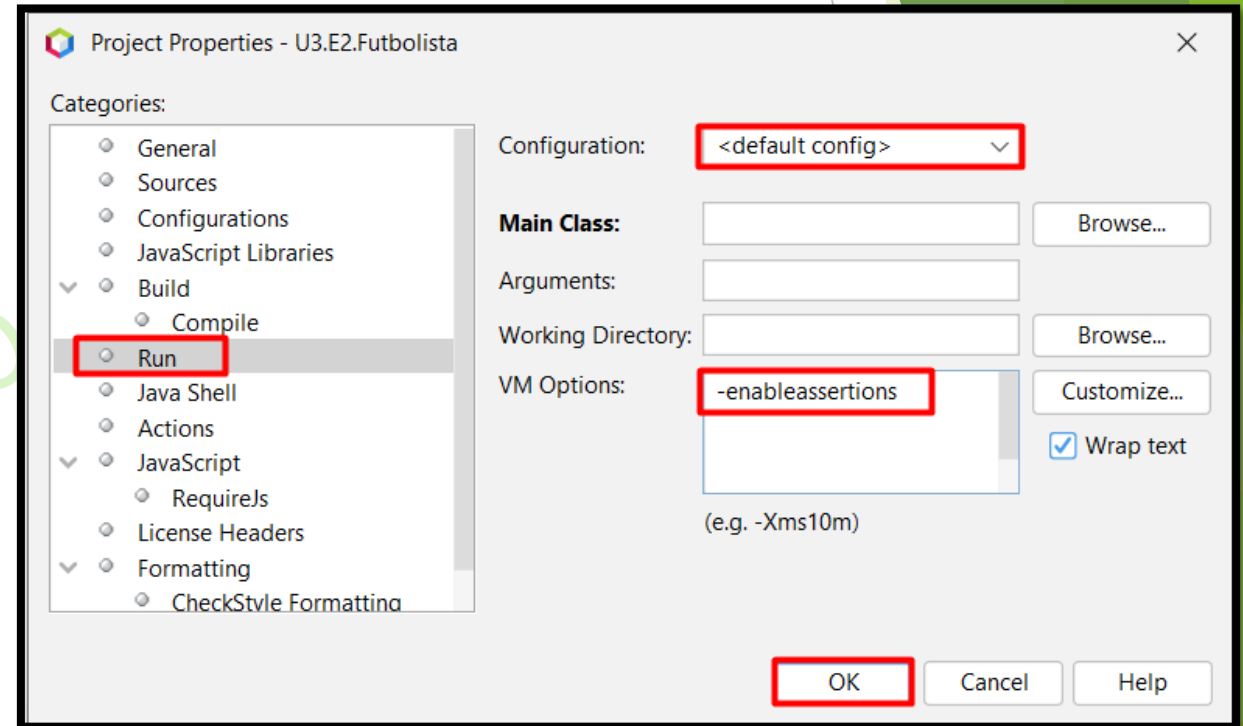
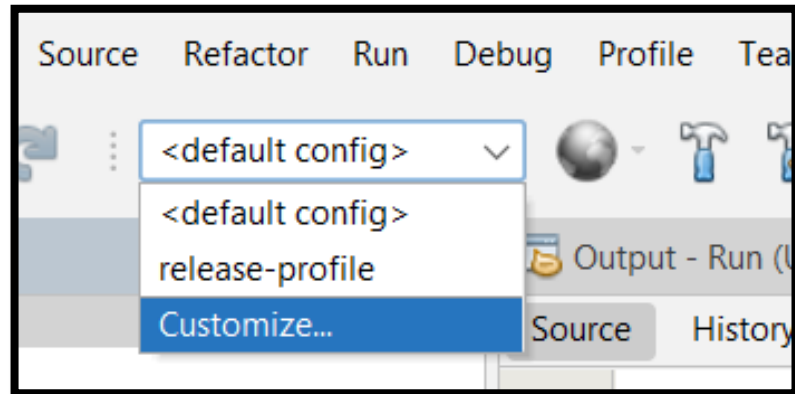
- **¿Por qué no ha funcionado nuestro aserto?** Porque la **MVJ** los tiene **desactivados por defecto**... Vamos a aprender a activarlos y después aprenderemos por qué se hace así.

WHY?

UD 3 - MEJORANDO NUESTRAS CLASES

5 - Asertos

- Para activarlos debemos modificar la **configuración por defecto de la MJV** en Netbeans:



- Si ahora volvemos a ejecutar nuestro *main* sí se produce el fallo:

```
Exception in thread "main" java.lang.AssertionError
    at es.marca.futbol.jugadores.Futbolista.marcaGol(Futbolista.java:21)
    at es.marca.futbol.PruebaFutbolista.main(PruebaFutbolista.java:11)
Command execution failed.
```

Pinchando en los enlaces nos lleva a la línea donde se produjo el fallo

UD 3 - MEJORANDO NUESTRAS CLASES

5 - Asertos

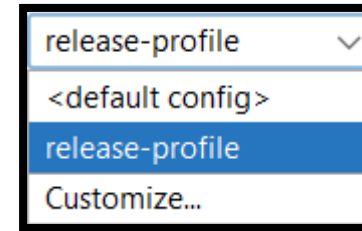
- ▶ Y ¿si esta herramienta es "tan buena" porque viene desactivada por defecto?
 - ▶ Decíamos que los **asserts** siguen la "filosofía" **FAIL FAST**, que nos ayuda en **la fase de desarrollo** de nuestra aplicación. En esta fase nuestro programa fallará muchas veces porque todavía no está maduro y los **asserts** nos ayudarán a detectar errores rápidamente.
 - ▶ Sin embargo, cuando nuestra aplicación se ha probado, no presenta fallos y se considera estable **se "libera"** (*release*) **pasando a una fase de "producción" o "explotación"**, es decir, se sube a un servidor para que preste un servicio a uno o varios clientes. En esta fase nos interesa que la aplicación presente una **ALTA DISPONIBILIDAD** y no se pare con cualquier fallo... Ahora ya **no nos sirven los asserts**. En este caso nos interesa que los fallos se registren en un fichero o una BD y que no se pare la aplicación. Este enfoque se le llama **FAIL SAFE** y para implementarlo se usan las excepciones que estudiaremos en la unidad 6.



UD 3 - MEJORANDO NUESTRAS CLASES

5 - Asertos

- ▶ Fíjate ahora que Netbeans nos proporciona otra configuración de la MVJ llamada "**release-profile**", que normalmente se asocia a la fase de producción, en la que no estarán activos los asserts.
- ▶ Netbeans nos permite alternar cómodamente entre ambas configuraciones mediante una lista desplegable:
- ▶ Hay muchas **librerías de asserts** que permiten escribir asertos de forma más legible y expresiva. Para saber más:



<https://www.adictosaltrabajo.com/2016/01/07/haz-tu-codigo-mas-fiabile-con-asserts/>

Realizar los ejercicios 3 y 4
del boletín

SOFTWARE
RELEASE

UD 3 - MEJORANDO NUESTRAS CLASES

6 - El modificador static

6 – El modificador static

- ▶ Este modificador ya lo hemos usado sin darnos mucha cuenta. Observa:

- ▶ **Math.PI**

- ▶ Es una propiedad estática definida dentro de la clase Math como:

```
public static final double PI = 3.1415....;
```

- ▶ **Math.random()**

- ▶ Es un método estático definido dentro de la clase Math como:

```
public static double random() {...}
```

- ▶ **¿Qué nos permite static?** al declarar una propiedad o un método como **estático entonces podremos usarlos directamente, sin tener que crear un objeto de dicha clase.**



STATIC

UD 3 - MEJORANDO NUESTRAS CLASES

6 - El modificador static

- Es decir, sería incorrecto hacer lo siguiente:

```
Math m = new Math();  
m.random();
```



- La clase Math funciona como una “caja de herramientas” es decir una colección de **propiedades o métodos estáticos** que tienen poca conexión entre sí, pero que sirven como "utilidades" a otras clases.
 - A este tipo de clases en las que todos los elementos son estáticos se les llaman “clases degeneradas” porque rompen con el principio de la POO de que “Todo es un objeto”.
 - Esta es la forma de uso más extendida del modificador static.



UD 3 - MEJORANDO NUESTRAS CLASES

6 - El modificador static

La clase `java.util.Arrays`

Esta clase es la caja de herramientas que brinda Java para trabajar con los arrays. **Arrays** ofrece muchos métodos estáticos útiles que nos permitirán:

- ▶ Ordenar un array (*sort*)
- ▶ Comparar dos arrays (*equals*)
- ▶ Copiar un array en otro de distinto tamaño (*copyOf*)
- ▶ Buscar un elemento en un array ordenado (*binarySearch*)
- ▶ Obtener una representación de texto del array (*toString*)
- ▶ ...

Podemos ver la documentación oficial de la clase aquí:

- ▶ <https://docs.oracle.com/javase/8/docs/api/java/util/Arrays.html>

Descarga el proyecto
U3.P3.PruebaArrays del
repositorio y estúdialo

UD 3 - MEJORANDO NUESTRAS CLASES

6 - El modificador static

Ordenación de un array (sort)

- ▶ A lo largo de la historia de la computación se han ido creando distintos algoritmos de ordenación de una colección de elementos. Los algoritmos de **inserción**, **burbuja** o "**primero el menor**" son algunos de los más conocidos. Siempre se estaba buscando un **algoritmo mejor que el anterior** (más rápido o que consuma menos memoria...).
- ▶ Para saber más, mira la siguiente lista de reproducción:
<https://www.youtube.com/watch?v=6GU6AGEWYJY&list=PLfBtpqIBIz7qyXI8TK8KPHYyIRVlvIFY8>
- ▶ A día de hoy el algoritmo llamado **Quicksort** es el más rápido (aunque consume más memoria que otros). Cuando usamos **Arrays.sort (...)** estamos usando este algoritmo (hoy en día la memoria es "barata")

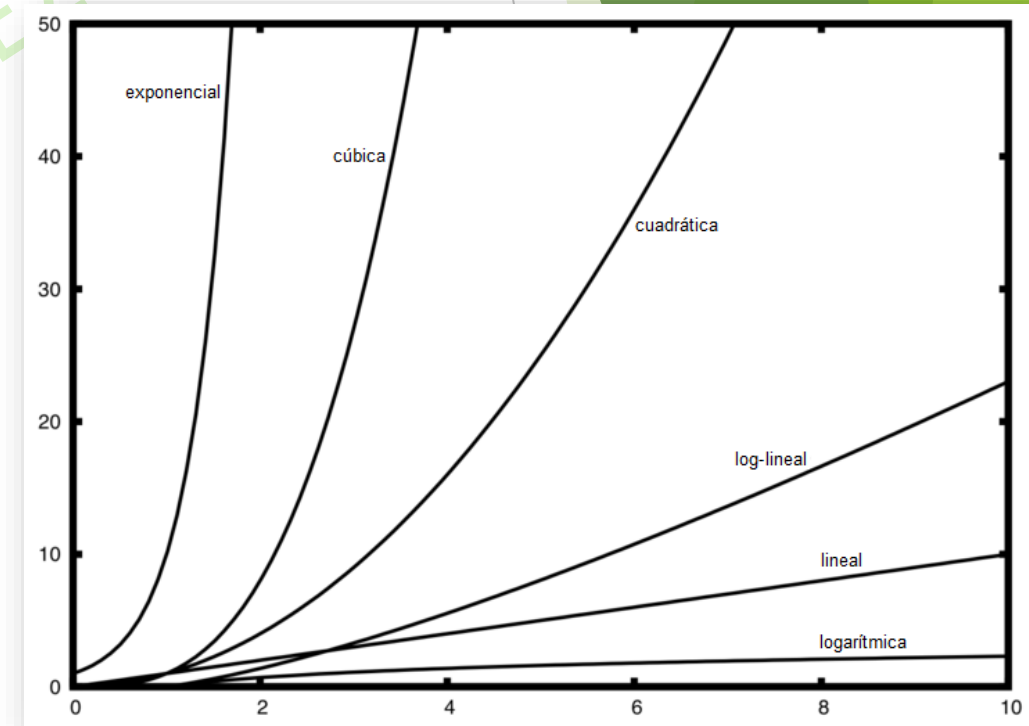


UD 3 - MEJORANDO NUESTRAS CLASES

6 - El modificador static

Buscar un elemento en un array ordenado (búsqueda binaria)

- ▶ Cuando tenemos un array desordenado de **n elementos** y queremos hacer una búsqueda, **en el peor de los casos** recorreremos el array completo y no encontramos el elemento. A este tipo de búsqueda se le llama **búsqueda secuencial**.
- ▶ Dado que, en el peor de los casos "pasaremos" por los n elementos se dice que la búsqueda secuencial posee un **“Orden de magnitud n” y se denota como $O(n)$** . Esto implica que el coste computacional de una búsqueda secuencial tiene un comportamiento **LINEAL**.
- ▶ Pero si el **array n elementos estuviera ordenado...** ¿podríamos aprovecharlo para hacer una búsqueda que no pase por todos los elementos, consiguiendo un coste computacional menor que $O(n)$?



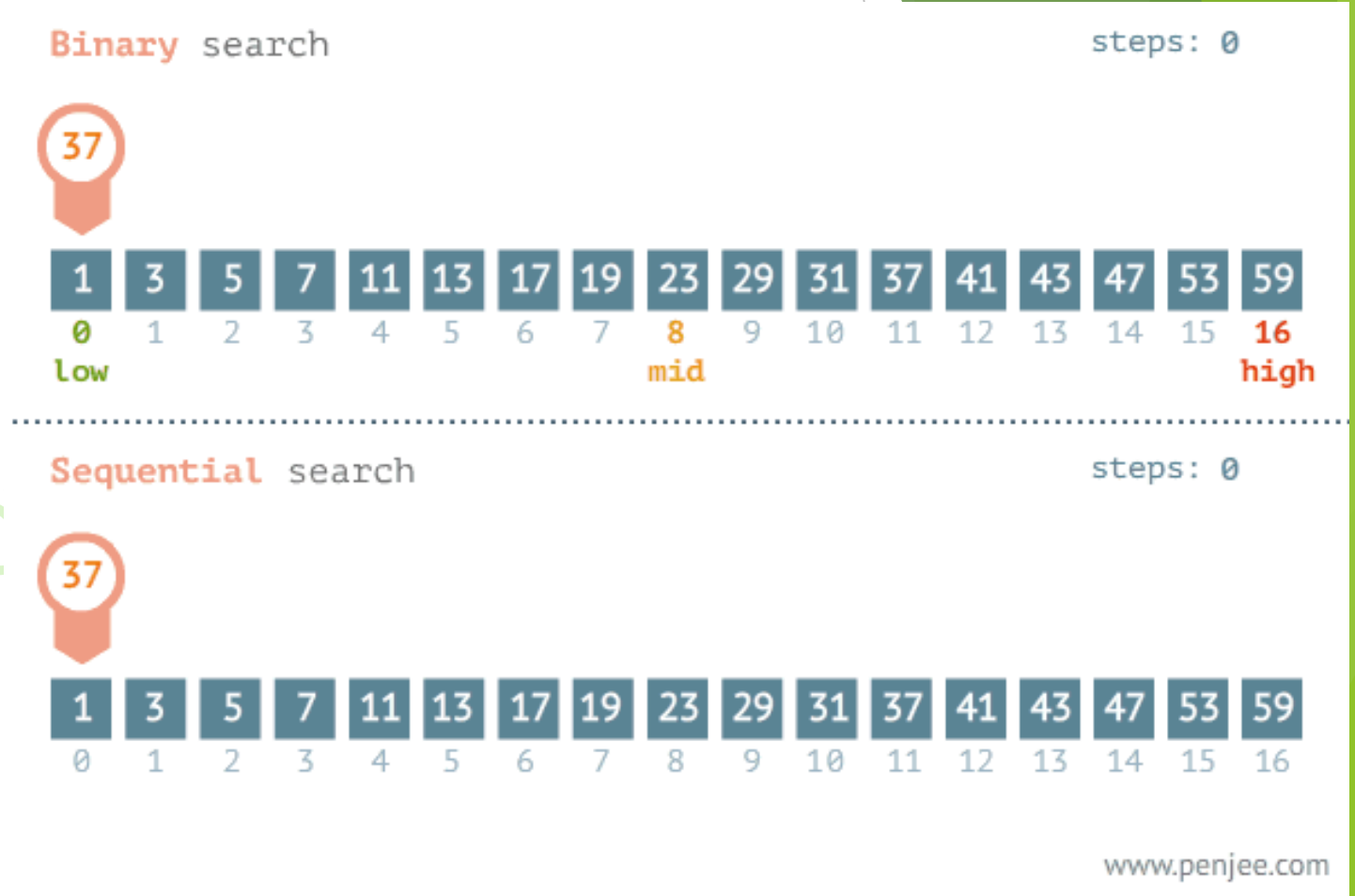
$$O(\log n) < O(n) < O(n^2) < O(n^3) < O(a^n)$$

UD 3 - MEJORANDO NUESTRAS CLASES

6 - El modificador static

- ▶ La lógica del algoritmo se muestra en el siguiente artículo:
<https://medium.com/@Emmitta/b%C3%BAsqueda-binaria-c6187323cd72>
- ▶ El método estático **binarySearch(...)** de la clase Arrays implementa esta búsqueda binaria y se obtiene menor coste computacional porque ahora tenemos un orden de magnitud de $O(\log n)$.

Realizar los ejercicios 5-7 del boletín



A partir de ahora si tenemos un array ordenado debemos usar el método `Arrays.binarySearch(...)` para hacer búsquedas porque es el más eficiente

UD 3 - MEJORANDO NUESTRAS CLASES

6 - El modificador static

Propiedades estáticas

Cuando aplicamos **static** a una propiedad se convierte en una **propiedad compartida** por todos los objetos de la clase. Observa:

```
public class U2DemoStatic {  
  
    public static void main(String[] args) {  
        Bombilla b1 = new Bombilla();  
        Bombilla b2 = new Bombilla();  
  
        System.out.println("El consumo actual es " + Bombilla.getConsumoTotalBombillas() + " W");  
  
        b1.setPotencia(100);  
        b2.setPotencia(60);  
        b1.encender();  
        b2.encender();  
  
        System.out.println("El consumo actual es " + Bombilla.getConsumoTotalBombillas() + " W");  
    }  
}
```



```
El consumo actual es 0 W  
El consumo actual es 160 W  
BUILD SUCCESSFUL (total time: 0 seconds)
```

*Este tipo de uso es mucho menos frecuente
que el de los métodos estáticos*

```
public class Bombilla {  
    // La propiedad estática consumoTotal será compartida entre todos  
    // los objetos. Es obligatorio inicializarla a la vez que se declara  
    public static int consumoTotal = 0;  
    public boolean encendida;  
    public int potencia;  
  
    public void encender() {  
        encendida = true;  
        consumoTotal = consumoTotal + this.potencia;  
    }  
  
    public void apagar() {  
        encendida = false;  
        consumoTotal = consumoTotal - this.potencia;  
    }  
  
    public static int getConsumoTotalBombillas() {  
        return consumoTotal;  
    }  
  
    public void setPotencia(int potencia) {  
        this.potencia = potencia;  
    }  
  
    public int getPotencia() {  
        return potencia;  
    }  
}
```

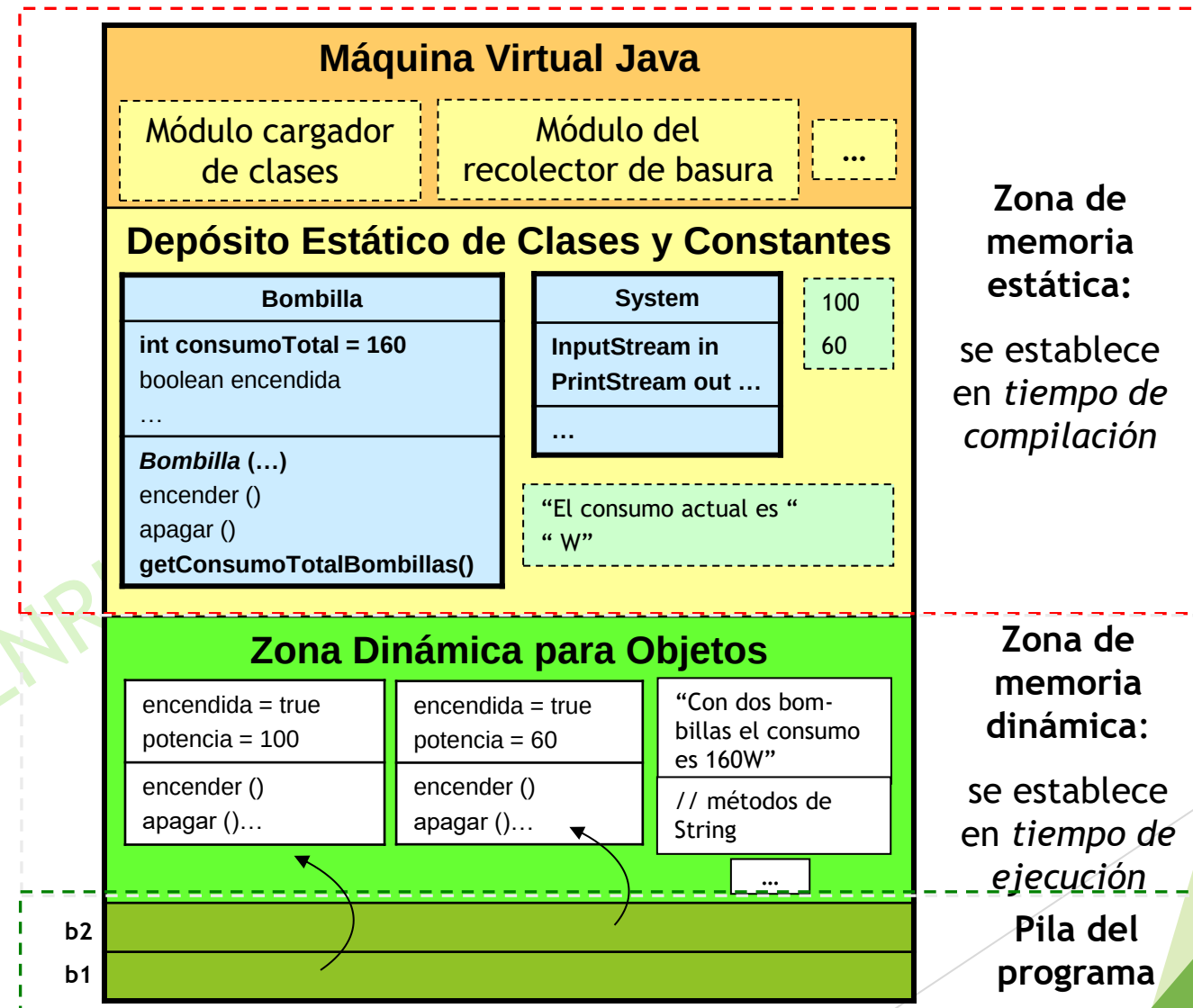
Descarga el proyecto
U3.P4.PropiedadesStatic.zip
y analiza el código

UD 3 - MEJORANDO NUESTRAS CLASES

6 - El modificador static

Como dato "curioso":

- ▶ Las propiedades y los métodos **static** se **almacenan en la zona estática de la memoria**. De ahí viene el nombre.
- ▶ Java organiza la memoria como se expresa en el diagrama siguiente:



UD 3 - MEJORANDO NUESTRAS CLASES

6 - El modificador static

¿Cómo usaremos static en este módulo profesional?

- ▶ Usaremos **static** para usar o crear clases tipo “caja de herramientas”.
- ▶ No lo usaremos para compartir propiedades entre objetos de una clase.
- ▶ Usaremos métodos estáticos para crear métodos auxiliares al método *main* que permitan hacer más “legible” nuestro código. Analiza el proyecto **U3.P5.MetodosAuxiliaresMain.zip**

- ▶ Ahora nuestro *main* se lee mejor con cosas como:

```
case 1:
    imprimirElementos(array);
    break;
```

- ▶ Observa como creamos un **método estático** que sirve como apoyo o auxilio al método *main*.

```
public static void imprimirElementos(int[] array) {
    for(int elem : array) {
        System.out.println(elem);
    }
}
```

UD 3 - MEJORANDO NUESTRAS CLASES

6 - El modificador static

¿Y qué pasa si se me olvida poner static en el método?

```
public void imprimirElementos(int[] array) {  
    for(int elem : array) {  
        System.out.println(elem);  
    }  
}
```



```
25         switch(opcion) {  
26             non-static method imprimirElementos(int[]) cannot be referenced from a static context  
27             ----  
28             (Alt-Enter shows hints)  
29             imprimirElementos(array);  
            break;
```

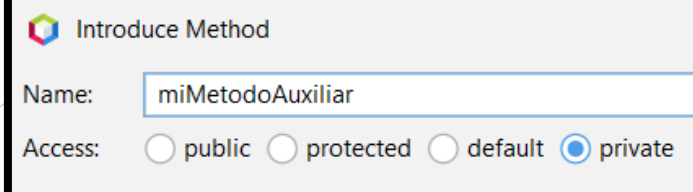
*Esto se produce porque main es un método **static** y solo puede invocar a métodos auxiliares que también sean **static***

Realizar el ejercicio 8 del boletín

¿Te atreves con el RETO del boletín?

Creando métodos auxiliares ayuda del IDE

Observa como el profesor/a refactoriza un método *main* sin métodos auxiliares con la opción: **Refactor -> Introduce -> Method...**



Introduce Method

Name:

Access: ☐ public ☐ protected ☐ default ☒ private

UD 3 - MEJORANDO NUESTRAS CLASES

Cierre de la unidad

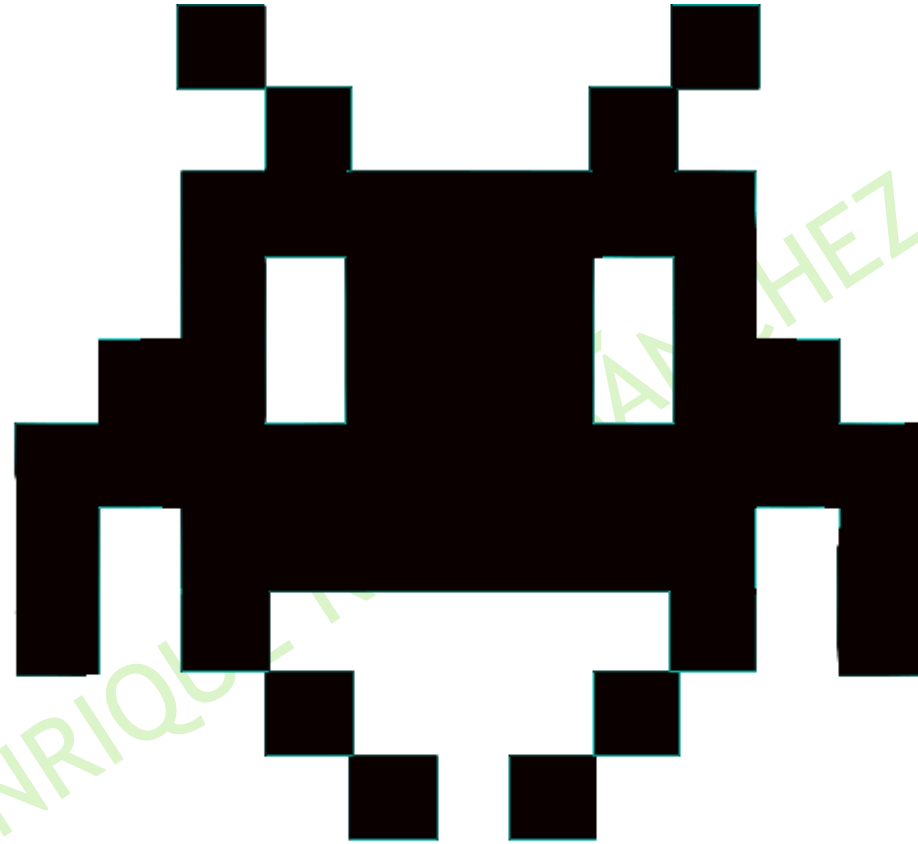
- ▶ En esta unidad hemos visto que en el mundo del software **TODO CAMBIA** y que necesitamos preparar nuestro código para que “**abraze el cambio**” y no luche contra él.
- ▶ Con el encapsulamiento, un buen diseño de los constructores y con los asertos conseguiremos que nuestras **clases sean más sólidas**.
- ▶ Con la **sobrecarga** de métodos podemos reusar el nombre de un método para distintos parámetros de entrada, así no "memorizamos" más cosas de la cuenta...
- ▶ Con **static** podemos usar y crear clases "tipo caja de herramientas" y crear métodos auxiliares para nuestro *main* haciendo más legible el código.
- ▶ Todas estas técnicas las seguiremos trabajando en las próximas unidades

No es la especie más fuerte la que sobrevive, ni la más inteligente, sino la que responde mejor al cambio



Charles Darwin

UD 3 - MEJORANDO NUESTRAS CLASES



The end screen